

Full Stack Practice Questions - 2

2 MARKS

1. # Components of Redux architecture -

- (i) Store: centralized container that holds the entire application state
- (ii) Actions: plain objects describing what changes should happen
- (iii) Reducers: functions that update state based on dispatched actions
- (iv) Dispatch: mechanism used to send actions to the Redux store
- (v) Middleware: handles asynchronous operations like API calls or logging

2. # Purpose of Context API -

- (i) Provides a way to share data across multiple components
- (ii) Eliminates prop drilling through deeply nested component structures
- (iii) Helps manage small scale global state inside React applications
- (iv) Improves communication between parent & deeply nested child components
- (v) Commonly used for themes, authentication state & user preferences

3. # Common ~~Responsive~~ Responsive Layout Techniques -

- (i) Flexbox for flexible alignment & distribution of UI components elements
- (ii) CSS grid for creating complex 2D responsive layouts
- (iii) Media queries to apply styles based on screen size
- (iv) Fluid layouts using percentage widths instead of fixed pixels
- (v) Responsive images & containers that scale across different devices.

4. # Common HTTP Methods -

- | | |
|--|---|
| (i) GET = used to retrieve data | (iv) DELETE = removes a resource from the server database |
| (ii) POST = used to send new data | (v) PATCH = partially updates specific fields |
| (iii) PUT = replaces or updates an existing resource | |

5. # Responsive UI Design Principles -

- (i) Mobile first design approach
- (ii) Flexible grid layouts that adjust according to screen size
- (iii) Media queries to adapt UI
- (iv) Scalable typography ensuring readable text
- (v) Responsive media elements like that resize properly

6. # Benefits of Centralized State Management -

- (i) Maintains a single source of truth
- (ii) Simplifies debugging by tracking all state changes centrally
- (iii) ~~Media queries to adapt UI across different screen sizes~~
- (iv) ~~Scalable~~ (iii) Ensures consistent data across multiple components & pages
- (iv) Makes app. logic more predictable & easy to maintain
- (v) Improves scalability

7. # Data Flow in Single Page Application -

- (i) User interaction triggers an event
- (ii) Component processes the event
- (iii) An API request may be sent to retrieve server data
- (iv) Server returns updated data
- (v) UI updates dynamically without a page reload.

8. # Standard Form Validation Rules -

- (i) Required field validation (can't be ~~non~~ empty)
- (ii) Email format validation (checks email structure)
- (iii) Min and Max length checks for usernames or passwords
- (iv) Numeric or date format validation for numbers & ~~valid~~ dates
- (v) Password strength validation (~~use~~ uppercase, digits or symbols)

9. # State Management in frontend Systems -

- (i) Maintains dynamic data used by diff. components
- (ii) Updates the UI whenever the app state changes
- (iii) Helps share data between multiple components
- (iv) Improves consistency of app behavior & UI rendering
- (v) Common tools include React state hooks, Redux & Context API

10. # Controlled VS Uncontrolled Components -

- | Controlled | Uncontrolled |
|--------------------------------|--------------------------------|
| (i) Managed by React state | (i) Managed by DOM |
| (ii) Uses use State | (ii) Uses ref to access values |
| (iii) Predictable data flow | (iii) Less react control |
| (iv) Easier validation | (iv) Simpler implementation |
| (v) React controls input value | (v) Useful for simple forms |

11. # Error handling in API consumption -

- (i) Use try-catch blocks to capture API ~~req~~ request errors
- (ii) Checks HTTP status codes to detect failed responses
- (iii) Handle network connectivity failures
- (iv) Display meaningful error messages
- (v) Log error information for debugging

12. # 2 advantages of Axios -
- (i) Automatically converts JSON responses into JS objects
 - (ii) Provides better error handling
 - (iii) Supports request & response interceptors
 - (iv) Built in timeout config for handling slow network requests
 - (v) Simplified syntax for sending HTTP requests

13. # Purpose of Reducers in Redux -
- (i) Handle dispatched actions sent from app components
 - (ii) Define how the app state should change
 - (iii) Return a new updated state without mutating previous state
 - (iv) Ensure predictable state updates through pure function logic
 - (v) Centralize state transition logic inside Redux architecture

14. # Characteristics of SPA -
- (i) Entire app loads using a single HTML page
 - (ii) Content updates dynamically using JS without page reloads
 - (iii) Uses frameworks like React, Angular or Vue
 - (iv) Communicates with backend services
 - (v) Provides faster navigation

15. # Advantages of Asynchronous API Calls -
- (i) Allows non blocking extensions execution
 - (ii) Keeps user interface responsive during back ground data fetching
 - (iii) Multiple API requests can run simultaneously
 - (iv) Improves overall performance
 - (v) Enhances user experience with faster loading times

16. # Working Principle of Fetch API -
- (i) Sends HTTP requests from browser
 - (ii) Returns a promise object
 - (iii) Uses .then() chaining or async/await syntax for handling responses
 - (iv) Converts response data into JSON or text format
 - (v) Allows handling of asynchronous network communication in JS

17. # Local vs Global Application State -

Local State

- (i) Managed inside component
- (ii) Used for UI data
- (iii) eg - input values
- (iv) Updated with react hooks
- (v) Limited scope

Global State

- (i) Shared across components
- (ii) Managed by Redux / Context
- (iii) Used for app wide data
- (iv) Accessible throughout app
- (v) Centralized state control

18. # Fetch vs Axios (Request handling) -

Fetch

- (i) Built in browser API
- (ii) Requires manual JSON parsing
- (iii) Uses promises
- (iv) Less automatic error handling
- (v) Lightweight implementation

Axios

- (i) External library
- (ii) Automatic JSON conversion
- (iii) Better error handling
- (iv) Supports interceptors
- (v) Easier syntax for requests

19. # Redux vs Context API -

Redux

- (i) Used for complex state
- (ii) Centralized state store
- (iii) Middleware support
- (iv) Predictable state updates
- (v) Suitable for large apps

Context API

- (i) Built into React
- (ii) Simpler state sharing
- (iii) No middleware support
- (iv) Less boilerplate code
- (v) Suitable for small apps

20. # Importance of Client side validation -

- (i) Detects incorrect user input before sending data to ~~user~~ server
- (ii) Improves user experience by ~~providing~~ providing instant feedback
- (iii) Reduces unnecessary server requests caused by invalid data
- (iv) Helps maintain better data quality in app
- (v) Prevents common input mistakes like empty or invalid fields

21. # Suitable UI elements for responsive forms -
- (i) Flexible input fields that resize according to screen width
 - (ii) Dropdown menus for selecting predefining options efficiently
 - (iii) Radio buttons & checkboxes for multiple selection inputs
 - (iv) Responsive buttons that adapt to different screen sizes
 - (v) Grid or flexbox layouts for proper form element alignment

22. # Role of APIs in SPAs -

- (i) Fetch dynamic data from backend servers in real time
- (ii) Sends user generated data to backend services for processing
- (iii) Acts as communication bridge between frontend & backend
- (iv) Enables dynamic UI updates without refreshing the page
- (v) Supports asynchronous data exchange in modern web applications

23. # Role of Breakpoints in Responsive UI -

- (i) Define screen width ranges for applying CSS styles
- (ii) Adjust layout for mobile, tablet, and desktop devices
- (iii) Improve usability & readability on different screen sizes
- (iv) Implemented using CSS media queries in responsive design
- (v) Helps developers optimize UI elements for various devices

24. # Single Page Application (SPA)

- (i) Web app that loads a single HTML page initially
- (ii) Dynamically updates content using JS and APIs
- (iii) Avoids full page reload during navigation between views
- (iv) Provides faster & smoother user interaction experience
- (v) Common frameworks include React, Angular & Vue

5 MARKS

1. # Challenges faced during responsive UI implementation -

- (i) Device fragmentation (different devices)
- (ii) Layout complexity (flexible layouts is difficult)
- (iii) Performance issues
- (iv) Cross browser compatibility
- (v) Testing difficulties
- (vi) Content prioritization

2. # State management using Redux (Sample app scenario) -

- (i) Install redux & configure a centralized store
- (ii) Define actions to describe state changes
- (iii) Create reducers that update the state
- (iv) Use dispatch() method to trigger state updates
- (v) Connect components using React Redux hooks
- (vi) eg - Managing shopping cart items

3. # Integration of REST APIs within a SPA -

- (i) SPA sends HTTP requests to backend APIs
- (ii) Use libraries like Fetch API or axios for sending API requests
- (iii) Data is retrieved from the server in JSON format
- (iv) Received data is stored in app state or redux store
- (v) UI components dynamically update based on API response
- (vi) Enables real time updates with page refresh

4. # ~~How~~ Workflow of a SPA with illustration -

- (i) User accesses the application & the main HTML file loads initially

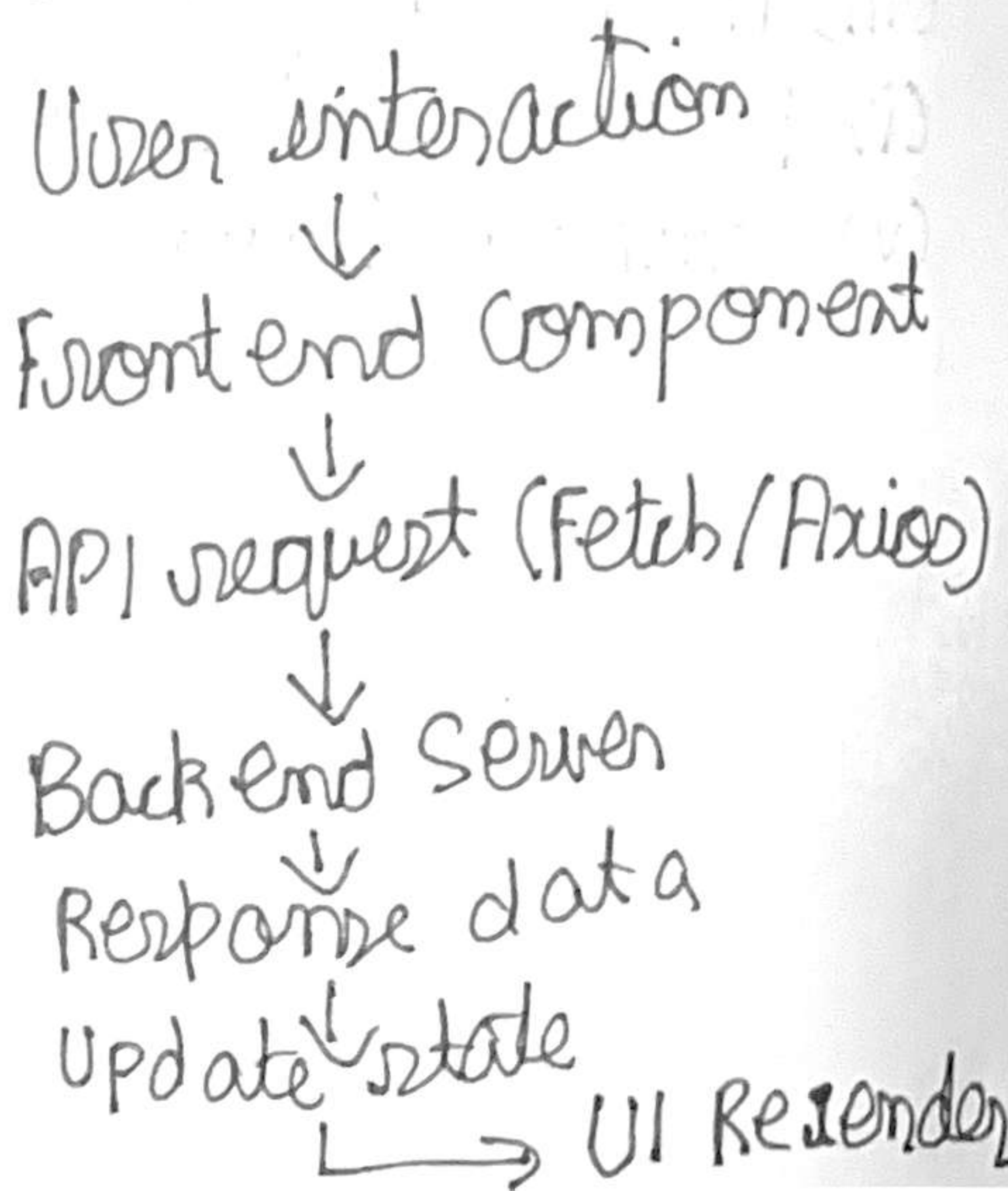
(ii) JS framework loads & renders components

(iii) User interaction triggers events like button clicks

(iv) App sends API requests to backend server for data

(v) Server returns data which updates app state

(vi) UI updates dynamically without performing a full page reload



5. # Different form validation strategies used in client side apps -

- (a) HTML validation:
 - Uses built in attributes like required, pattern & minlength
 - Simple & quick to implement
 - Limited customization & flexibility
 - (b) JS validation:
 - Custom validation logic using JS functions
 - Allows complex validation rules
 - Provides better control over error messages
 - (c) Framework based validation:
 - Uses libraries like React Hook form
 - Simplifies form state management & validation logic
- Suitable for large & complex forms

6. # API error handling techniques in frontend apps -

- (i) Uses try catch blocks when handling async API calls
- (ii) Check HTTP response status codes
- (iii) Display meaningful error messages
- (iv) Implement retry mechanisms
- (v) Log errors for debugging & monitoring
- (vi) Provide fallback UI when data loading fails

7. # Context API for managing global state in a sample use case -

- (i) Create a context object using `React.createContext()`
- (ii) Wrap the app with context provider component
- (iii) Share global data such as user authentication or theme settings
- (iv) Access context values inside components using `useContext` hook
- (v) Update shared state through provider functions
- (vi) Eliminates prop drilling across deeply nested components

8. # Responsive UI principles to adapt an app across multiple devices -

- (i) Use flexible grid systems like CSS Grid / Flexbox
- (ii) Implement media queries for different screen breakpoints
- (iii) Design layouts using mobile first approach
- (iv) Ensure images & media scale correctly
- (v) Adjust typography & spacing for readability
- (vi) Test UI across desktop, tablet & mobile

9. # Illustrate the form handling & validation process -

- (i) Form inputs are managed using Component state or hooks
- (ii) User inputs trigger onChange events
- (iii) Validation rules check input data
- (iv) Errors are displayed next to relevant input fields
- (v) Form submission sends validated data to backend APIs
- (vi) Libraries like React Hook form simplify this process

10. # Impact of poor state management on app performance -

- (i) Causes unnecessary component re-renders
- (ii) Leads to inconsistent data across multiple components
- (iii) Makes debugging & maintenance more difficult
- (iv) Increased complexity of ~~app logic~~ app logic & data flow
- (v) Can cause UI glitches & unpredictable behaviour
- (vi) Reduces scalability

11. # API Consumption Using Fetch or Axios -

- (i) Send API request using Fetch API or Axios library
- (ii) Use GET method to retrieve data
- (iii) Handle responses using Promises or async/await syntax
- (iv) Convert response data into JSON format
- (v) Display data dynamically within UI components

12. # Redux vs Context API with suitable use cases -

< Already done in UNIT-1 >

Redux
Use case: large e-commerce platforms or enterprise dashboards
(suitable for apps with heavy data sharing)

Context API: Sharing data like themes, language, or authentication
(not suitable for highly complex state management scenarios)

10 MARKS

1. # Responsive UI frameworks for compatibility across multiple devices -
- (i) Bootstrap provides pre built responsive grid systems & mobile first components
 - (ii) Tailwind CSS provides utility first classes that help create flexible layouts
 - (iii) Material UI provides responsive components optimized for modern devices
 - (iv) Frameworks support breakpoints for mobile, tablet & desktop screens
 - (v) Built in responsive utilities ensure consistent UI across browsers & devices
 - (vi) Frameworks reduce development time while maintaining cross device compatibility

2. # Design and validate complex forms with responsive UI considerations -
- (i) Use responsive layouts such as CSS grid or Flexbox
 - (ii) Organize inputs into logical groups for better usability
 - (iii) Implement valid client side validation rules
 - (iv) Display validation errors clearly near corresponding input fields
 - (v) Ensure form elements adjust properly for mobile, tablet & desktop screens
 - (vi) Use libraries like React Hook form for complex form handling

3. # Design & Analyze & Optimize API consumption strategies for high performance SPAs -
- (i) Use pagination or lazy loading to fetch data in smaller chunks
 - (ii) Use asynchronous API calls to prevent blocking the UI
 - (iii) Implement caching mechanisms to reduce repeated API requests
 - (iv) Optimize API calls using debouncing or throttling techniques
 - (v) Handle API errors efficiently with retry mechanisms & fallback UI
 - (vi) Minimize network requests by combining related API calls when possible

4. # Comprehensive frontend solution -
- (i) Use a JS framework like React to build a SPA
 - (ii) Implement routing using React Router for navigation without page reloads
 - (iii) Manage app state using Redux or Context API
 - (iv) Integrate REST APIs to fetch & update backend data dynamically
 - (v) Implement form handling with validation using React Hook forms
 - (vi) Apply responsive design principles using CSS grid, flexbox, and media queries

5. # Evaluate effectiveness of Redux versus Context API in large scale apps -

- (a) Redux:
- (i) Suitable for large scale apps with complex state management needs
 - (ii) Provides centralized store ensuring predictable state updates
 - (iii) Supports middleware for handling asynchronous operations like API calls
 - (iv) Offers powerful debugging tools such as Redux Dev tools.
 - (v) Improves scalability & maintainability for enterprise level apps

- (b) Context API:
- (i) Designed for simpler state sharing across components
 - (ii) Requires less setup & minimal boilerplate code
 - (iii) Suitable for small or medium sized app
 - (iv) Works well for features like theme management or authentication
 - (v) May cause performance issues in very large applications

6. # Complete SPA app architecture -

- (i) Frontend layer built using framework such as React, Angular, or Vue
- (ii) Routing system handles navigation without refreshing the page
- (iii) State management layer implemented using Redux or Context API
- (iv) API layer communicates with backend services using Fetch or Axios
- (v) Backend server processes requests & returns data in JSON format
- (vi) UI components dynamically update based on app state changes

